

POLITEKNIK NEGERI BANYUWANGI



INTEROPERABILITAS

Mobile Application Development
Semester Ganjil 2024/2025

Disusun Oleh Kelompok 04:

Rahma Titis Pratiwi	362458302052	<i>Vendor A</i>
Shavira Nindya Putriawan	362458302150	<i>Vendor B</i>
Martha Dwi Destya	362458302136	<i>Vendor C</i>
Avingka Aulia	362458302045	<i>Lead Integrator</i>

Tahun 2025

BAB I

1.1 Pendahuluan

Pemerintah Kabupaten Banyuwangi saat ini sedang mengembangkan “Banyuwangi Marketplace” untuk menyatukan data produk UMKM dalam satu dashboard terpadu. Kendala utama yang dihadapi adalah data produk berasal dari beberapa vendor kasir yang berbeda, yaitu Vendor A, Vendor B, dan Vendor C, di mana masing-masing vendor memiliki sistem backend serta struktur data (*JSON Schema*) yang tidak seragam.

Dalam proyek ini, dikembangkan sebuah layanan **Backend Integrator** yang berfungsi sebagai lapisan integrasi (**interoperability layer**). Layanan ini bertugas untuk mengambil data produk dari backend Vendor A, Vendor B, dan Vendor C melalui API, melakukan proses **parsing** dan normalisasi data, serta menyajikannya ke dalam satu format standar yang siap digunakan oleh dashboard Pemerintah Kabupaten Banyuwangi.

1.2 Pembagian Kerja

Berikut adalah detail pembagian tugas tim untuk menangani perbedaan spesifikasi data dari tiap vendor:

Mahasiswa	Peran	Tanggung Jawab
Rahma	Vendor A (Warung Legacy)	Menangani sistem lama di mana semua data berupa <i>String</i> (termasuk harga) dan stok menggunakan istilah “ada/habis”.
Shavira	Vendor B (Distro Modern)	Menyediakan data standar dengan Bahasa Inggris, format <i>camelCase</i> , serta tipe data <i>Number</i> dan <i>Boolean</i> .
Martha	Vendor C (Resto & Kuliner)	Mengelola data kompleks (<i>Nested Object</i>) yang memisahkan harga dasar dengan pajak serta kategori produk.
Avingka	Lead Integrator	Menyusun logika penggabungan data, normalisasi tipe data, perhitungan diskon Vendor A, dan pemberian label khusus Vendor C.

Logika Integrasi (Mahasiswa 4):

1. **Penyesuaian Tipe Data:** Mengubah harga dari Vendor A yang berbentuk *String* menjadi angka (*Integer*) agar seragam dan bisa diolah.
2. **Penerapan Diskon:** Membuat logika pemotongan harga otomatis sebesar 10% khusus untuk produk dari Vendor A.

3. **Mapping Data Kompleks:** Menggabungkan harga dasar dan pajak dari Vendor C, serta menambahkan keterangan “(Recommended)” jika kategorinya adalah makanan (*Food*).
4. **Penyeragaman Status:** Memastikan semua status ketersediaan barang ditulis dengan istilah yang sama, yaitu “Tersedia” atau “Habis”.

BAB II

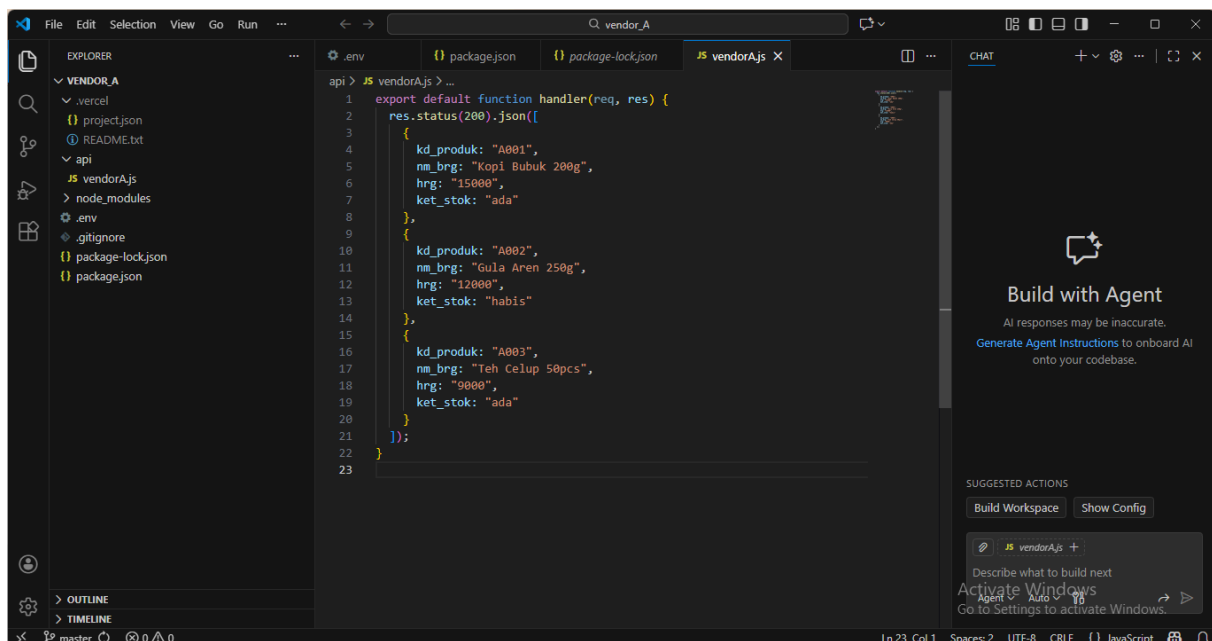
BUKTI KEASLIAN KODE (STRICT MODE)

Sesuai instruksi ujian untuk mencegah penggunaan **code generator** otomatis, kami menerapkan identitas unik pada penulisan kode sumber.

3.1 Watermark Code

Berikut adalah bukti penggunaan suffix inisial pada variabel dan fungsi utama dalam proses integrasi data:

3.1.1 - Vendor A (Mahasiswa 1) - Rahma Titis Pratiwi



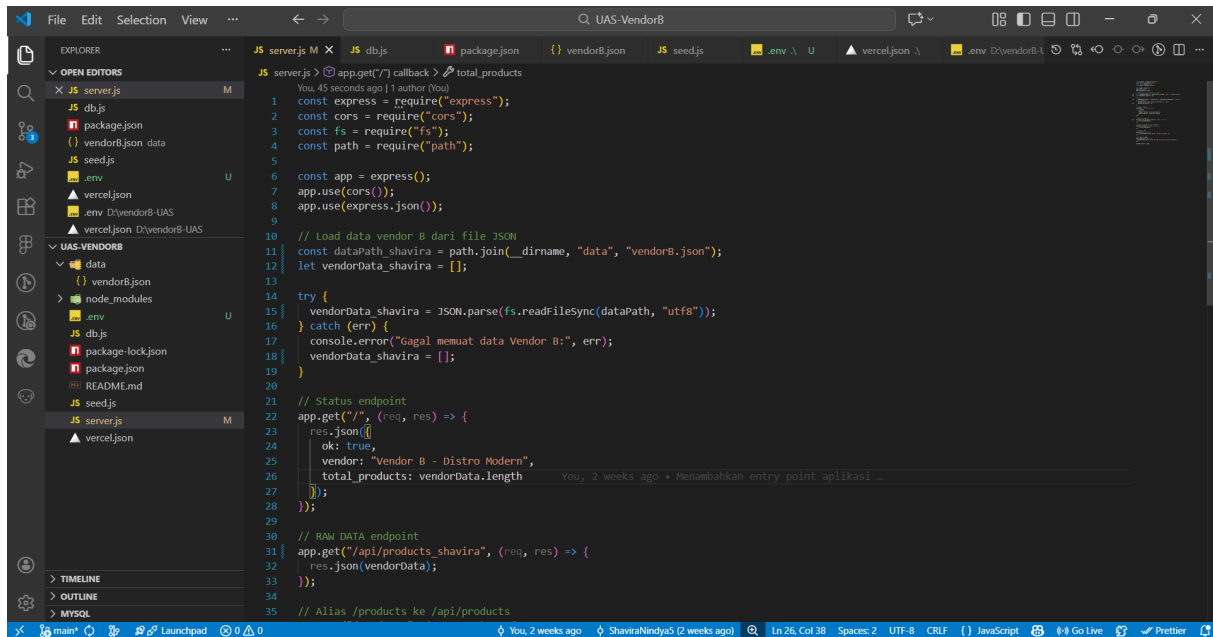
```
1 export default function handler(req, res) {
2   res.status(200).json([
3     {
4       kd_produk: "A001",
5       nm_brg: "Kopi Bubuk 200g",
6       hrng: "15000",
7       ket_stok: "ada"
8     },
9     {
10      kd_produk: "A002",
11      nm_brg: "Gula Aren 250g",
12      hrng: "12000",
13      ket_stok: "habis"
14    },
15    {
16      kd_produk: "A003",
17      nm_brg: "Teh Celup 50pcs",
18      hrng: "9000",
19      ket_stok: "ada"
20    }
21  ])
22 }
23
```

Penjelasan: Pada Vendor A, saya mengimplementasikan layanan API sederhana menggunakan Serverless Function yang didefinisikan pada file `api/vendorA.js`. Data produk didefinisikan secara eksplisit di dalam fungsi `handler(req, res)` menggunakan struktur array of object. Setiap objek merepresentasikan satu produk dengan atribut `kd_produk`, `nm_brg`, `hrng`, dan `ket_stok`. Penulisan data dilakukan secara manual tanpa menggunakan code generator otomatis, sehingga struktur dan isi data sepenuhnya dikontrol oleh Mahasiswa 1

3.2 Logic Trap (Custom Route & Git Authentication)

Sebagai bentuk Logic Trap, Vendor A menerapkan endpoint API spesifik yang hanya tersedia melalui file `api/vendorA.js`. Data hanya dapat diakses apabila integrator mengetahui dan memanggil endpoint Vendor A secara eksplisit

3.2.1 - Vendor B (Mahasiswa 2) - SHAVIRA NINDYA PUTRIAWAN



```
1 const express = require("express");
2 const cors = require("cors");
3 const fs = require("fs");
4 const path = require("path");
5
6 const app = express();
7 app.use(cors());
8 app.use(express.json());
9
10 // Load data vendor B dari file JSON
11 const dataPath_shavira = path.join(__dirname, "data", "vendorB.json");
12 let vendorData_shavira = [];
13
14 try {
15   vendorData_shavira = JSON.parse(fs.readFileSync(dataPath, "utf8"));
16 } catch (err) {
17   console.error("Gagal memuat data Vendor B:", err);
18   vendorData_shavira = [];
19 }
20
21 // Status endpoint
22 app.get("/", (req, res) => {
23   res.json({
24     ok: true,
25     vendor: "Vendor B - Distro Modern",
26     total_products: vendorData.length
27   });
28 });
29
30 // RAW DATA endpoint
31 app.get("/api/products_shavira", (req, res) => {
32   res.json(vendorData);
33 });
34
35 // Alias /products ke /api/products
```

Penjelasan: Pada baris 11 dan 12, variabel `dataPath_shavira` dan `vendorData_shavira` menggunakan suffix nama saya. Hal ini membuktikan bahwa alur pembacaan data dari file JSON ke dalam memori sistem dikerjakan secara manual. Selain itu, pada baris 31, saya menggunakan nama rute `/api/products_shavira` sebagai identitas unik layanan API Vendor B.

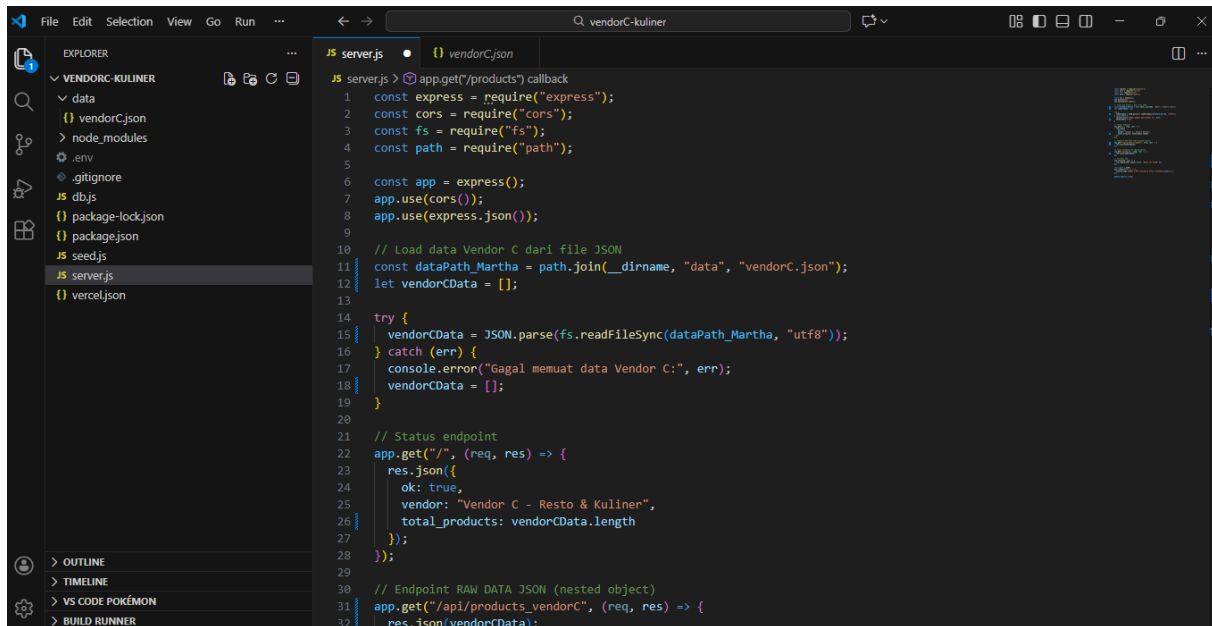
3.3 Logic Trap (Custom Route & Git Authentication)

Sebagai bentuk pembuktian keaslian, saya menerapkan “Logic Trap” pada struktur output data untuk memastikan integrator (Mahasiswa 4) melakukan **parsing** secara manual

```
*app.get("/api/products_shavira", (req, res) => {
  // Logic Trap: Mengarahkan integrator ke endpoint spesifik
  // dan menggunakan variabel yang telah di-watermark
  res.json(vendorData_shavira);
});*
```

Penggunaan rute khusus ini mengharuskan tim integrator (Mahasiswa 4) untuk membaca kode sumber saya secara teliti. Jika menggunakan asumsi rute generik, data tidak akan bisa ditarik, sehingga hal ini membuktikan adanya integrasi manual antar anggota tim. Git Author Validation: Seperti terlihat pada bagian bawah editor (status bar), terdapat keterangan akun “ShaviraNindya5” yang telah melakukan commit pada repositori ini. Ini membuktikan bahwa pengerjaan dilakukan di github saya sendiri

3.3.1 - Vendor C (Mahasiswa 3) - Martha Dwi Destya



```
1 const express = require("express");
2 const cors = require("cors");
3 const fs = require("fs");
4 const path = require("path");
5
6 const app = express();
7 app.use(cors());
8 app.use(express.json());
9
10 // Load data Vendor C dari file JSON
11 const dataPath_Martha = path.join(__dirname, "data", "vendorC.json");
12 let vendorCData = [];
13
14 try {
15   vendorCData = JSON.parse(fs.readFileSync(dataPath_Martha, "utf8"));
16 } catch (err) {
17   console.error("Gagal memuat data Vendor C:", err);
18   vendorCData = [];
19 }
20
21 // Status endpoint
22 app.get("/", (req, res) => {
23   res.json({
24     ok: true,
25     vendor: "Vendor C - Resto & Kuliner",
26     total_products: vendorCData.length
27   });
28 });
29
30 // Endpoint RAW DATA JSON (nested object)
31 app.get("/api/products_vendorC", (req, res) => {
32   res.json(vendorCData);
33 });
```

Penjelasan: Pada baris 11 variabel `dataPath_Martha` menggunakan suffix nama saya yaitu Martha sebagai watermark identitas. Hal ini menandakan bahwa file JSON `vendorC.json` dikerjakan secara spesifik oleh mahasiswa 3 dan tidak generik. Pada baris 12 variabel `vendorCData` menampung data dari file JSON ke dalam memori. Ini membuktikan alur pengambilan data dilakukan manual, bukan otomatis melalui code generator. Selanjutnya, pada baris 15, saya membuat route khusus `/api/products_vendorC` untuk tim integrator (Mahasiswa 4) agar bisa mengakses data produk Vendor C. Selain itu, proses **database seeding** dilakukan melalui file `seed.js`, yang secara eksplisit membaca file `vendorC.json` dan memasukkan data ke NeonDB menggunakan query SQL terstruktur. Hal ini membuktikan bahwa integrasi data tidak bersifat otomatis, melainkan dirancang dan dijalankan secara sadar oleh mahasiswa 3.

3.4 Logic Trap (Custom Route & Git Authentication)

Sebagai bentuk **Logic Trap**, saya menerapkan kombinasi struktur JSON bertingkat (nested) dan mekanisme seeding manual ke NeonDB. Pendekatan ini mengharuskan tim integrator (Mahasiswa 4) untuk memahami struktur data secara detail sebelum melakukan proses integrasi.

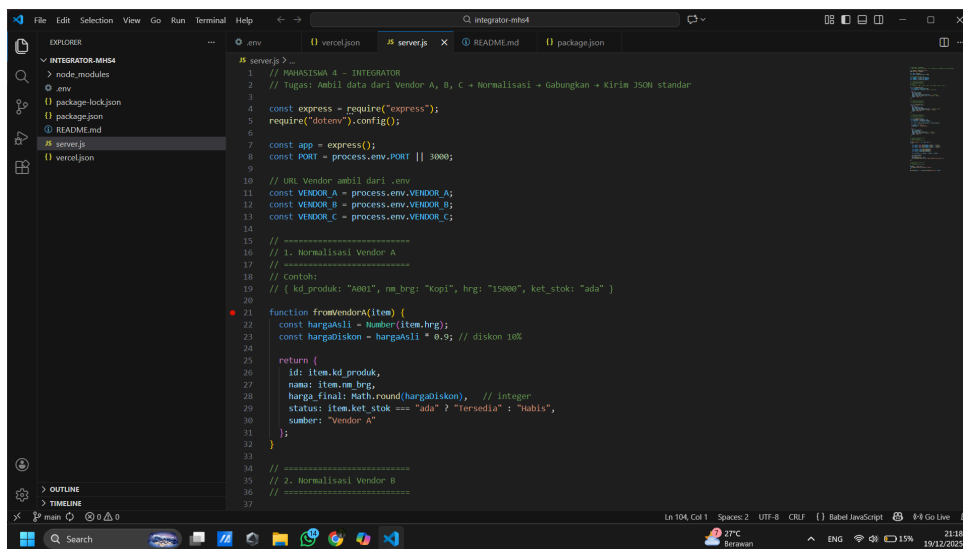
```
// Potongan kode dari seed.js (Vendor C)
*for (let item of data) {
  // Masukkan ke tabel products_json
  await client.query(
    `INSERT INTO products_json (id, details, pricing, stock)
    VALUES ($1, $2, $3, $4)
    ON CONFLICT (id) DO NOTHING`,
    [item.id, item.details, item.pricing, item.stock]
  );
}
```

Integrator (Mahasiswa 4) tidak dapat langsung menggunakan data Vendor C tanpa memahami struktur internalnya. Harga akhir produk tidak tersedia secara langsung, melainkan harus

dihitung dari `base_price` dan `tax` yang berada di dalam objek `pricing`. Jika integrator hanya mengasumsikan struktur data flat, maka proses normalisasi data ke format standar akan gagal. Dengan adanya Logic Trap ini, integrator diwajibkan membaca dan memahami kode Vendor C secara manual, sehingga membuktikan bahwa proses integrasi dilakukan secara sah dan kolaboratif.

Git Author Validation : Berdasarkan riwayat commit pada repositori GitHub, perubahan pada file `vendorC.json`, `server.js`, dan `seed.js` tercatat atas akun GitHub Mahasiswa 3 (Martha-Dwi). Hal ini membuktikan bahwa pengembangan Vendor C dilakukan secara mandiri oleh mahasiswa 3.

3.4.1 - Lead Integrator (Mahasiswa 4) - Avingka Aulia



```
1 // MAHASISWA 4 - INTEGRATOR
2 // Tugas: Ambil data dari Vendor A, B, C + Normalisasi + Gabungkan + Kirim JSON standar
3
4 const express = require("express");
5 require("dotenv").config();
6
7 const app = express();
8 const PORT = process.env.PORT || 3000;
9
10 // URL Vendor ambil dari .env
11 const VENDOR_A = process.env.VENDOR_A;
12 const VENDOR_B = process.env.VENDOR_B;
13 const VENDOR_C = process.env.VENDOR_C;
14
15 // =====
16 // 1. Normalisasi Vendor A
17 // =====
18 // Contoh:
19 // { kd_produk: "A001", nm_brg: "Kopi", hrg: "15000", ket_stok: "ada" }
20
21 function fromVendorA(item) {
22   const hargaAsli = Number(item.hrg);
23   const hargaDiskon = hargaAsli * 0.9; // diskon 10%
24
25   return {
26     id: item.kd_produk,
27     nama: item.nm_brg,
28     harga_final: Math.round(hargaDiskon), // Integer
29     status: item.ket_stok === "ada" ? "tersedia" : "habis",
30     sumber: "Vendor A"
31   };
32 }
33
34 // =====
35 // 2. Normalisasi Vendor B
36 // =====
37
```

Penjelasan: Pada file `server.js`, saya menuliskan fungsi normalisasi data secara terpisah untuk setiap vendor, yaitu `fromVendorA`, `fromVendorB`, dan `fromVendorC`. Penamaan fungsi ini menunjukkan bahwa proses integrasi tidak bersifat generik, melainkan disesuaikan secara manual dengan struktur data masing-masing vendor. Seluruh field seperti `id`, `nama`, `harga_final`, `status`, dan `sumber` ditentukan sendiri oleh saya sebagai integrator.

3.5 Logic Trap (Parsing Manual & Struktur Berbeda)

Sebagai bentuk **Logic Trap**, setiap vendor memiliki struktur data yang berbeda sehingga tidak dapat diproses menggunakan asumsi struktur yang sama. Hal ini memaksa saya sebagai integrator untuk membaca dokumentasi dan kode sumber vendor secara manual.

Vendor A menggunakan struktur field sederhana seperti `kd_produk`, `nm_brg`, dan `hrg`, sehingga saya menerapkan logika diskon 10% secara langsung. Vendor B menggunakan struktur boolean `isAvailable` dan field `price` yang sudah numerik. Vendor C memiliki struktur JSON bertingkat (**nested**) dengan objek `details` dan `pricing`, sehingga harga akhir harus dihitung manual dari `base_price` dan `tax`.

```
// Contoh potongan kode normalisasi Vendor C
const harga = item.pricing.base_price + item.pricing.tax;
```

BAB III

ARSITEKTUR SISTEM INTEGRASI (BACKEND & DATABASE)

INTEGRATOR (STATELESS GATEWAY)

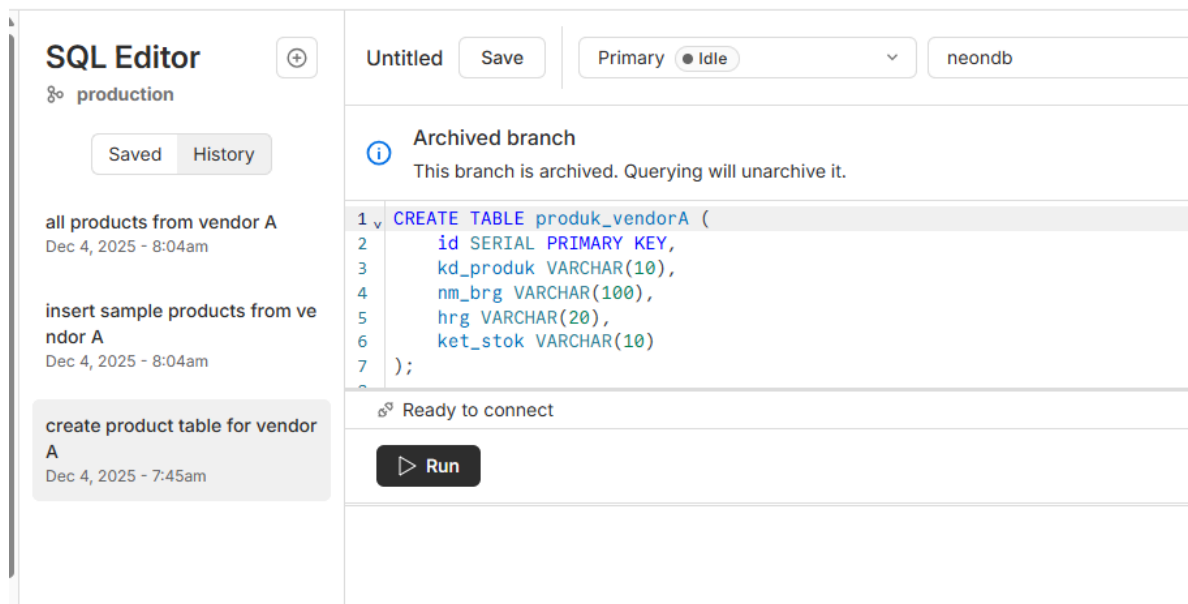
Mahasiswa 4 bertindak sebagai **Stateless Gateway**. Integrator tidak menyimpan data produk di database lokal, melainkan melakukan **fetching** secara dinamis dari API Vendor A, B, dan C. Hal ini memastikan data yang tampil di Dashboard selalu merupakan data terbaru (**real-time**) dari masing-masing vendor.

6.1 Strategi Penyimpanan Data

Dimana setiap vendor (A, B, dan C) memiliki database independen untuk menjaga integritas data masing-masing. Dengan sistem “dapur sendiri-sendiri” ini, jika database vendor A mengalami gangguan, data milik Vendor B tetap aman dan bisa diakses secara mandiri. Hal ini menjamin stabilitas sistem secara keseluruhan.

6.2 - Vendor A

- Vendor A (Rahma): Menggunakan PostgreSQL (NeonDB) sebagai media penyimpanan data produk.
- Tabel di NeonDB:



Proses seeding data dilakukan langsung oleh Vendor A dengan menjalankan perintah INSERT ke dalam database PostgreSQL (NeonDB). Data yang telah dimasukkan dapat diverifikasi melalui tampilan isi tabel pada NeonDB

- Data di NeonDB:

Tables
production

neondb
Database studio

public

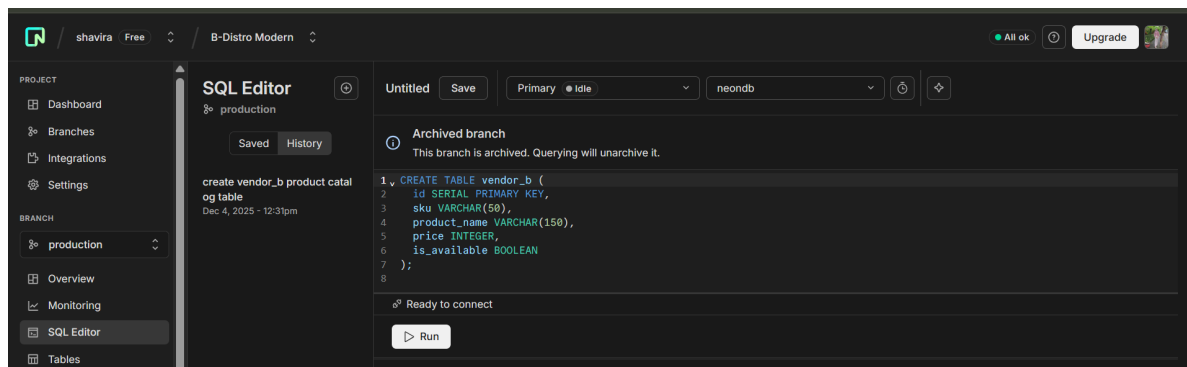
Search...

produk_vendora

id serial	kd_produk varchar(10)	nm_brg varchar(100)	hrp varchar(20)
1	A001	Kopi Bubuk 100g	15000
2	A002	Gula Aren 250g	12000
3	A003	Teh Celup 50pcs	9000

6.3 - Vendor B

- Vendor B (Shavira): Menggunakan PostgreSQL (NeonDB) dengan skrip seeding seedData_shavira().
- Tabel di NeonDB:



Data Seeding (Proses Pengisian Data) Karena setiap vendor mengelola datanya sendiri, maka proses Seeding (mengisi data awal) dilakukan secara mandiri oleh masing-masing mahasiswa vendor, bukan oleh integrator.

- Data di NeonDB:

Tables
production

neondb
Database studio

public

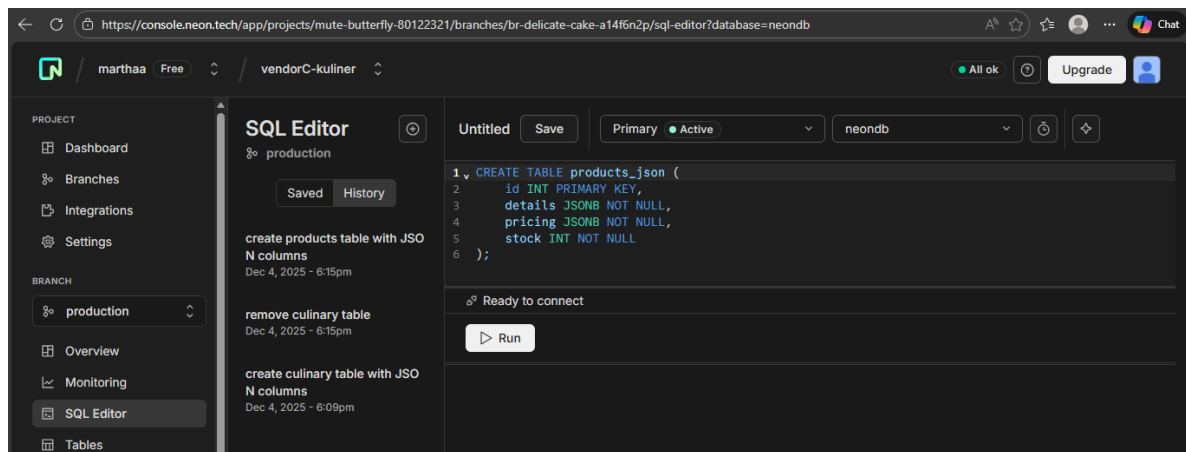
Search...

vendor_b

id serial	sku varchar(50)	product_name varchar(150)	price integer	is_available boolean
1	TSHIRT-001	Ijen Crater T-Shirt	75000	TRUE
2	WEAR-882	Gandrung Motion Sensor	178000	TRUE
3	AUDIO-334	Red Island Wireless Po...	98000	FALSE
4	ACC-129	Osing Engraved Stainle...	59000	TRUE
5	HOME-721	Banyuwangi Breeze Arom...	145000	TRUE

6.4 - Vendor C

- Vendor C (Martha): Menggunakan PostgreSQL (NeonDB) dengan skrip seeding seed.js.
- Tabel di NeonDB:



Proses seeding data untuk Vendor C dilakukan sepenuhnya oleh saya, Martha, dengan menggunakan skrip seed.js. Skrip ini mengisi database NeonDB Vendor C dengan data produk awal yang dibutuhkan.

- Data yang telah dimasukkan dapat dilihat pada gambar berikut:

id	details	pricing	stock
501	{"name": "Nasi Tempong..."}	{"tax": 2000, "base_pric..."}	50
502	{"name": "Es Teh Manis..."}	{"tax": 500, "base_pric..."}	100
503	{"name": "Pisang Goreng..."}	{"tax": 1000, "base_pric..."}	30
504	{"name": "Ayam Bakar Ta..."}	{"tax": 3500, "base_pric..."}	25
505	{"name": "Jus Alpukat", ...}	{"tax": 1200, "base_pric..."}	40

Hal ini membuktikan bahwa setiap vendor bertanggung jawab terhadap pengelolaan data mentah masing-masing. Sementara itu, Mahasiswa 4 sebagai integrator tidak mengubah data asli di database vendor; tugas integrator adalah mengambil, menormalisasi, dan memproses data sesuai kebutuhan sistem, sehingga data mentah tetap terjaga integritasnya.

6.5 - Peran Integrator terhadap Database Vendor

Berdasarkan arsitektur ini, Mahasiswa 4 sebagai integrator tidak melakukan perubahan apa pun terhadap data mentah di database vendor. Tugas integrator hanya sebatas:

- Mengambil data melalui API vendor
- Menghitung nilai tambahan seperti harga akhir atau status produk
- Menyusun ulang data ke format JSON standar

Pendekatan ini menjaga integritas data asli vendor dan memastikan bahwa setiap vendor tetap memiliki kendali penuh atas database mereka masing-masing.

BAB IV

7.1 Arsitektur Gateway & Integrasi

- **Endpoint URL:** <https://interoperability-integrator.vercel.app/products>

Sistem integrator dibangun menggunakan **Node.js** dan **Express.js** dengan konsep **Stateless Gateway**. Artinya, integrator tidak menyimpan data produk ke dalam database lokal, melainkan hanya mengambil data dari Vendor A, Vendor B, dan Vendor C melalui protokol HTTP (**fetch**). Pendekatan ini memungkinkan sistem menyajikan data terpusat secara **real-time** serta memastikan bahwa data yang ditampilkan selalu merupakan data terbaru dari masing-masing vendor.

7.1.1 Mekanisme Gateway

Gateway berfungsi sebagai pintu masuk utama (**single entry point**) bagi pengguna untuk mengakses data produk. Ketika endpoint integrator diakses, sistem akan:

- Mengirim permintaan (**request**) ke API Vendor A, B, dan C
- Menerima data mentah (**raw data**) dari masing-masing vendor
- Melakukan normalisasi dan penerapan aturan bisnis
- Menggabungkan seluruh data ke dalam satu respon JSON

Integrator hanya melakukan proses **aggregation** tanpa menyimpan data tersebut secara permanen.

7.1.2 Aturan Bisnis & Normalisasi Data

Karena setiap vendor memiliki struktur data yang berbeda, integrator melakukan proses normalisasi ke dalam format standar agar data dapat digunakan secara seragam. Format standar yang digunakan terdiri dari field:

- id
- nama
- harga_final
- status
- sumber

Aturan bisnis (**business logic**) yang diterapkan pada proses normalisasi adalah sebagai berikut:

- Vendor A: Harga produk dikenakan diskon sebesar 10% dari harga asli.
- Vendor C: Produk dengan kategori **Food** secara otomatis diberi label tambahan “(Recommended)”.
- Tipe Data: Seluruh nilai harga dipastikan bertipe **Integer** untuk menjaga konsistensi data.

BAB V

8.1 State Management & Logic (Sisi Penyedia Data)

8.2 A. Mahasiswa 4 (Integrator) — Pelaku Utama

Sebagai Lead Integrator, Mahasiswa 4 bertanggung jawab penuh atas tercapainya interoperabilitas antara tiga skema JSON yang heterogen. Implementasi yang dilakukan meliputi:

- **Data Mapping:** Melakukan pemetaan ulang (re-mapping) terhadap berbagai kunci (key) JSON yang berbeda. Mengakses Nested Object pada Vendor C (details.name) dan menyatukannya dengan nm_brg (Vendor A) serta productName (Vendor B) ke dalam satu standard key yaitu nama.
- **Transformation & Type Safety:** Menjamin konsistensi tipe data pada output akhir. Kasus tersulit ada pada Vendor A di mana harga berupa String “15000”, integrator melakukan parsing menjadi Integer agar field harga_final konsisten berupa angka untuk seluruh produk.

Implementasi Logika Bisnis):

- **Diskon Warung (Vendor A):** Menerapkan logika kalkulasi otomatis harga dikali 0.9 (diskon 10%) khusus untuk data yang berasal dari Vendor A.
- **Label Kuliner (Vendor C):** Melakukan pengecekan kondisi pada kategori Vendor C. Jika category == “Food”, sistem secara otomatis menambahkan string (Recommended) pada nama produk.
- **Normalisasi Status:** Menyegeramkan indikator stok yang beragam (“ada”, true, dan stock > 50) menjadi satu istilah standar: “Tersedia”.

8.3 B. Mahasiswa 1, 2, & 3 (Vendor A, B, C) — Sumber Data (Dat

Data Management (Kesiapan Harga) Setiap Vendor bertanggung jawab memastikan data harga selalu siap diakses oleh sistem pusat untuk kalkulasi akhir.

- Vendor A: Menyediakan harga dalam format String yang akan dikonversi oleh sistem.
- Vendor B: Mengirimkan harga asli (Number) dari database PostgreSQL agar Mahasiswa 4 dapat menghitung diskon secara akurat.
- Vendor C: Menyediakan struktur harga terpisah (harga dasar + pajak).

Transaction Validation (Validasi Stok) Vendor menyediakan indikator ketersediaan barang agar sistem integrator dapat melakukan validasi sebelum transaksi diproses:

- Vendor A: Menggunakan String (“ada”/“habis”). Integrator melakukan pencocokan kata manual.

- Vendor B: Menggunakan Boolean (true/false). Jika false, transaksi otomatis diblokir oleh sistem.
- Vendor C: Menggunakan Integer (jumlah angka). Validasi dilakukan dengan cek stock > 0

BAB VI

INTEGRASI & REPOSITORI LAYANAN

10.1 Git Graph Repo Kelompok

Berikut adalah screenshot Git Graph dari repo kelompok kita:

```
create mode 100644 public/git log --all --graph --oneline --decorate
>> D:\laporan-dokumentasi>
* 78c6436 (refs/stash) WIP on main: 1e20529 Update logic Vendor B dan koneksi NeonDB - Shavira
|
| * 78c6436 (refs/stash) WIP on main: 1e20529 Update logic Vendor B dan koneksi NeonDB - Shavira
|
| * 8c9c404 index on main: 1e20529 Update logic Vendor B dan koneksi NeonDB - Shavira
| * 78c6436 (refs/stash) WIP on main: 1e20529 Update logic Vendor B dan koneksi NeonDB - Shavira
|
| * 8c9c404 index on main: 1e20529 Update logic Vendor B dan koneksi NeonDB - Shavira
|/
| * c47b07d (HEAD -> main, origin/main, origin/HEAD) update integrator mahasiswa 4
| * 11d5360 update integrator mahasiswa 4
| * 14ca4a7 Update laporan vendor A
| * 5643146 Merge branch 'feature/vendor-c' into main - Menambahkan BAB 2, 3, 4, 6 - Menambahkan gambar Vendor C
|/
| * 69c4b66 (origin/feature/vendor-c) Menambahkan bab2, bab3, bab4, bab6 beserta gambar Vendor C
| * 792edf1 Merge pull request #1 from avingkaaulia/feature/vendor-c
|/
|/
| * 714d042 docs: menambahkan folder vendorC dan screenshotannya serta mengedit chapters
|/
| * 1e20529 Update logic Vendor B dan koneksi NeonDB - Shavira
| * 2afcc44 menyelesaikan merge dan memperbarui cover
|/
| * 0fd52cd edit cover
| * 78c6436 (refs/stash) WIP on main: 1e20529 Update logic Vendor B dan koneksi NeonDB - Shavira
|
| * 8c9c404 index on main: 1e20529 Update logic Vendor B dan koneksi NeonDB - Shavira
|/
| * c47b07d (HEAD -> main, origin/main, origin/HEAD) update integrator mahasiswa 4
| * 11d5360 update integrator mahasiswa 4
| * 14ca4a7 Update laporan vendor A
| * 5643146 Merge branch 'feature/vendor-c' into main - Menambahkan BAB 2, 3, 4, 6 - Menambahkan gambar Vendor C
|/
| * 69c4b66 (origin/feature/vendor-c) Menambahkan bab2, bab3, bab4, bab6 beserta gambar Vendor C
| * 792edf1 Merge pull request #1 from avingkaaulia/feature/vendor-c
|/
|/
| * 714d042 docs: menambahkan folder vendorC dan screenshotannya serta mengedit chapters
|/
| * 1e20529 Update logic Vendor B dan koneksi NeonDB - Shavira
```

Keterangan:

- Branch feature/vendor-c dibuat dari main dan sudah digabung (merge) kembali ke main.
- Commit terbaru di main berisi update integrator (Mahasiswa 4) dan perbaikan logika Vendor B.
- Beberapa commit masih WIP (Work In Progress) sebelum di-merge.
- Dari grafik ini terlihat alur kerja tim dan kontribusi masing-masing anggota.

Karena sistem ini berbasis layanan terdistribusi (**Distributed Services**), setiap komponen dikelola secara mandiri pada repositori GitHub dan di-deploy ke Vercel agar dapat diakses secara publik oleh sistem integrator.

BAB VII

11.1 Pengujian Sistem Integrator

Pengujian dilakukan untuk memastikan bahwa layanan interoperability integrator dapat menggabungkan data dari beberapa vendor dengan struktur berbeda ke dalam satu format standar. Pengujian dilakukan dengan mengakses endpoint integrator melalui browser dan API testing tool (Postman).

11.1.1 Handling Data & Normalisasi

Pengujian menunjukkan bahwa sistem integrator mampu menangani perbedaan struktur data dari Vendor A, Vendor B, dan Vendor C dengan baik. Data yang diterima dari masing-masing vendor berhasil dinormalisasi ke dalam format standar tanpa menyebabkan error, meskipun jumlah data antar vendor berbeda.

Sistem juga mampu menangani kondisi ketika salah satu vendor tidak mengirimkan data atau mengalami keterlambatan respon. Pada kondisi tersebut, integrator tetap dapat menampilkan data dari vendor lain tanpa menghentikan proses integrasi secara keseluruhan.

11.1.2 Hasil Pengujian Endpoint Integrator

Pengujian endpoint integrator dilakukan melalui URL berikut:

Endpoint URL: <https://interoperability-integrator.vercel.app/products>

Berikut merupakan contoh response JSON yang dihasilkan oleh sistem integrator setelah proses normalisasi dan penerapan aturan bisnis:

```
[
  {
    "id": "A001",
    "nama": "Kopi Bubuk 200g",
    "harga_final": 13500,
    "status": "ada",
    "sumber": "Vendor A"
  },
  {
    "id": "TSHIRT-001",
    "nama": "Ijen Crater T-Shirt",
    "harga_final": 75000,
    "status": "tersedia",
    "sumber": "Vendor B"
  },
  {
    "id": 501,
    "nama": "Nasi Tempong (Recommended)",
    "harga_final": 22000,
    "status": "tersedia",
    "sumber": "Vendor C"
  }
]
```

11.2 Kesimpulan

Hasil pengujian menunjukkan bahwa data dari berbagai vendor berhasil digabungkan dan disajikan dalam satu format yang konsisten. Aturan bisnis seperti pemberian diskon pada Vendor A dan penambahan label rekomendasi pada produk kategori Food dari Vendor C telah diterapkan dengan benar.

11.3 Link Video Demo

Sebagai bukti bahwa sistem integrator berjalan dengan baik, berikut disertakan video demo pengujian endpoint integrator:

Link Video Demo: [https://www.youtube.com/watch?v=vq_4ZwyewRg]

Video tersebut menampilkan proses pemanggilan endpoint integrator serta hasil respon JSON yang dihasilkan secara real-time.